# Sistemas Operativos

Clase Práctica 1

# Shell o intérprete de comandos

- Programa que se encarga de ejecutar las órdenes del usuario, para esto:
  - Muestra un ***prompt*** para que el usuario ingrese una secuencia de comandos y argumentos.
  - Busca el ejecutable del comando requerido (en un conjunto de ubicaciones posible)
  - Crea un nuevo proceso hijo, le carga la imagen del ejecutable con los argumentos, y espera a que finalize.
  - Vuelve a pedir otro comando (paso 1).
- También provee algunos comandos implementados directamente por la shell, incluso un lenguaje para realizar ***shell-scripting*** (scripts ejecutados por la shell).
- En los entornos gráficos, se tiene un emulador gráfico, que permite configurar colores, sesiones, personalización, scroll, perstañas, etc. Se los conoce como **Consolas** o **Terminales**.

# Shell o intérprete de comandos

- Algunas shells:
  - **Thompson shell (sh)**: Primer shell de Unix, 1971, desarrollada por Ken Thompson.
  - **Bourne shell (sh)**: Unix version 7, 1979, Stephen Bourne, se convirtió de facto en Unix.
  - **GNU Bash (Bourne again shell, bash)**: Versión GNU de bourne, de facto en Linux.
  - **Z Shell (zsh)**:
  - **C Shell (csh)**:
- Algunas terminales:
  - **Gnome Terminal / Konsole**: Las por defecto de Gnome / KDE
  - **Guake / Yakuake / Tilda**: Consolas desplegables semitransparentes, imitando al Quake.
  - **Terminator**: Subdivide la pantalla en varias terminales, puede enviar comandos a todos a la vez.

# Shell o intérprete de comandos

- Los comandos tienen un **nombre** y **parámetros / opciones / flags** que pueden ser requeridos u opcionales. Las opciones y flags en general tienen un nombre corto y uno largo.
  - **-v, --version**: Muestra la versión del comando.
  - **-h, --help**: Muestra información de ayuda sobre el uso del comando.
- En general, si un comando finaliza correctamente no muestra confirmación en pantalla. Sólamente muestra mensajes si se produjo un error o alerta (warning).
- Los nombres de los comandos son ***case-sensitive***.

# Comandos básicos

- **man (manual)**: Documentación “online” de un comando (si el comando implementa). Tiene varias secciones, en gral usaremos la 2 y 3.
- **echo**: Muestra texto por pantalla.
- **clear**: Borra la pantalla.
- **sleep**: Bloquea el proceso un tiempo determinado y termina.
- **head, tail**: Muestra por pantalla las primeras/últimas n líneas de un archivo.
- **more, less**: Paginan la salida por pantalla (para hacer scrolling)

# Comandos de información

- **date**: muestra (o setea) la fecha y hora actual del sistema.
- **cal**: muestra un calendario (cal 2021, cal 08 2021, …)
- **who**: muestra información sobre los usuarios logged-in en el sistema.
- **whoami**: muestra el nombre de usuario del usuario actual.
- **free**: Muestra información sobre memoria libre y usada en el sistema.
- **id**: Muestra información (los ids) sobre usuario actual y grupo.
- **uname**: Muestra información del sistema (uname -a)
- **whereis <comando>**: Ubicación del comando, su fuente y su man-page.
- **which <comando>**: Nos muestra qué comando resuelve la shell (en caso que haya varios con el mismo nombre, en diferentes lugares del $PATH.
- **top / ps / pstree**: Información de procesos (lo veremos mas adelante)

# Comandos para navegación / manejo de archivos

- **pwd** (Path to Working Directory): Muestra el directorio de trabajo.
- **ls** (list): Muestra el contenido del directorio actual.
- **cd** (change directory): Se posiciona en un determinado directorio.
- **cp** (copy), **mv** (move), **rm** (remove): Copia, mueve, elimina archivos / directorios.
- **mkdir** (make dir), **rmdir** (remove dir): Crea, elimina un directorio.
- **cat** (concat): concatena el contenido de archivo(s) por salida estándar.
- **touch**: cambia la fecha de modificación o último acceso de un archivo.
- **wc** (word count): Cuenta palabras, renglones, bytes de un archivo.

# Símbolos especiales

- Hay algunos símbolos especiales:
  - **~** : Directorio home del usuario. (e.g. **ls ~/Downloads** )
  - **.** (punto): Directorio actual. (e.g. **cp /tmp/*.txt  .**  // Copia todos los txt de /tmp al dir actual)
  - **..** (punto punto): Directorio padre. (e.g. **cd ../../labo1** )
- Comodines (wildcards)
  - ***** : reemplaza cualquier caracter.
  - **?** : reemplaza un solo caracter.
  - **[caracteres]** : reemplaza un grupo de caracteres.
  - **[!caracteres]** : reemplaza un grupo que no está en caracteres.

# Variables de entorno

- Variables ‘globales’, con información para los comandos o el usuario:
  - **$PATH**: Contiene la lista de ubicaciones donde se buscan los ejecutables.
  - **$SHELL**: Contiene la shell que tiene asignada el usuario.
  - **$HOME**: Contiene el path del directorio home del usuario.
  - **$?**: Contiene el valor de retorno del último comando ejecutado (OK: **0**, Not OK: **!= 0**)
- **printenv**: Muestra el valor de las variables de entorno.
- **echo $?**: Muestra el valor de retorno del último comando ejecutado.

# Comandos en background

- **cmd &**: Si terminamos el comando con ampersand, en vez de esperar a que el hijo finalize, el comando ejecuta en background y la terminal muestra el prompt para pedir el siguiente comando.
- **jobs**: Muestra los procesos en background / foreground y su estado.
- **fg**: Pone un determinado proceso en foreground.
- **bg**: Pone un determinado proceso en background.
- **ctrl + c** : Termina el proceso en foreground.
- **ctrl + z** : Pausa / detiene el proceso en foreground. (Se resume con fg/bg)

# Concatenando comandos

- **cmd1; cmd2; cmd3**: Ejecuta cmd1, luego cmd2, luego cmd3.
- **cmd1 && cmd2** : Ejecuta cmd2 sólamente si cmd1 finaliza CON éxito.
- **cmd1 || cmd2** : Ejecuta cmd2 sólamente si cmd1 finaliza SIN éxito.

Cada proceso tiene asignado como mínimo 3 archivos (file descriptors):

- **0: Standard input (stdin)**. Entrada estándar, en gral teclado.
- **1: Standard output (stdout)**. Salida estándar, en gral pantalla.
- **2: Standard error (stderr)**. Salida de errores, en gral pantalla.

# Concatenando comandos

Podemos armar flujos de trabajo, concatenando comandos y haciendo que la salida de un comando sea la entrada del siguiente y así sucesivamente. Esto se hace con la operación de **pipe**.

- **cmd1 | cmd2** : ejecuta cmd1 y usa su salida como entrada para cmd2.
  - Ej: **cat texto.txt | wc**  cat imprime el contenido del archivo y esa entrada la consume wc, que cuenta la cantidad de palabras. Por pantalla solo veremos el resultado de wc.

# Redirección de entradas y/o salidas.

- **cmd > result.txt** : Ejecuta cmd, y su salida es redirigida al archivo result.txt.
  - **cat file1 file2 file3 > result.txt**
- **cmd >> result.txt** : Idem, pero hace append del archivo.
- **cmd < input.txt** : Ejecuta cmd, tomando como entrada input.txt.
- Para redirigir más de una salida a la vez, usar el file descriptor:
  - **cmd 1> result.txt 2> errors.txt** // ejecuta cmd, su salida estandar va a result.txt, su salida de errores va a errors.txt
- Mas comandos:
  - **grep**: filtra texto por expresiones regulares, y muestra los resultados por pantalla.
  - **find**: Busca archivos por nombre, fecha, author, etc.